

Part 4: State-Space Control - Advanced Techniques

This section covers the design and implementation of state-space controllers, observers, and optimal control (LQR).

Python Scripts Needed

- `practiceFinalSim4.py`: The main simulation script for Part 4.
 - `ssi_controller.py`: Implementation of State-Space Integral (SSI) control.
 - `ssi_dist_obs_controller.py`: SSI combined with a Disturbance Observer.
 - `lqr_controller.py`: Optimal LQR implementation.
 - `params.py`: Linearization matrices (A, B, C) and controller gains (K, L).
-

4.1 - 4.2: State-Space Integral Control (SSI)

Description

Augment the system with an integrator to achieve zero steady-state error.

What to do:

1. **Define Augmented System:** Formulate the matrices:

$$A_{aug} = \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix}, \quad B_{aug} = \begin{bmatrix} B \\ 0 \end{bmatrix}$$

2. **Pole Placement:** Choose desired poles for the closed-loop system (e.g., $p = [-10, -11, -12]$). Use `scipy.signal.place_poles` to find gains K and k_i .
 3. **Implement SSI:** Update `ssi_controller.py` with your gains.
-

4.3 - 4.5: Observers & Disturbance Rejection

Description

Implement a state observer and a disturbance observer to estimate unmeasured states and reject constant disturbances.

What to do:

1. **Full-State Observer:** Design observer gains L so that observer poles are 2-10x faster than controller poles.

2. **Disturbance Observer:** Augment the observer with a disturbance state d :

$$\begin{bmatrix} \dot{\hat{x}} \\ \dot{\hat{d}} \end{bmatrix} = \begin{bmatrix} A & B \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \hat{x} \\ \hat{d} \end{bmatrix} + \dots$$

3. **Reject Disturbance:** In `ssi_dist_obs_controller.py`, use the estimate $\hat{d}$ in your control law: $u = u_e - K\hat{x} + k_i x_i - \hat{d}$.
-

4.6 - 4.7: LQR Optimal Control

Description

Use Linear Quadratic Regulator (LQR) to find the optimal balance between performance and control effort.

What to do:

1. **Cost Function Weights:** Define Q (state penalty) and R (control penalty) in `params.py`.
2. **Solve ARE:** In `lqr_controller.py`, the `control.lqr` function solves the Algebraic Riccati Equation to find K_{opt} .
3. **Run Simulation:** Run `python practiceFinalSim4.py` and analyze the difference between pole placement and LQR.